Solution: Permutations

Problem Statement

Given a set of distinct numbers, find all of its permutations.

Permutation is defined as the re-arranging of the elements of the set. For example, {1, 2, 3} has the following six permutations:

{1, 2, 3}
{1, 3, 2}
{2, 1, 3}
{2, 3, 1}
{3, 1, 2}
{3, 2, 1}

If a set has 'n' distinct elements it will have n!n! permutations.

Example 1:

```
Input: [1,3,5]
Output: [1,3,5], [1,5,3], [3,1,5], [3,5,1], [5,1,3], [5,3,1]
```

Solution

This problem follows the `Subsets` pattern and we can follow a similar **Breadth First Search** (**BFS**) approach. However, unlike `Subsets`, every permutation must contain all the numbers.

Let's take the example-1 mentioned above to generate all the permutations. Following a BFS approach, we will consider one number at a time:

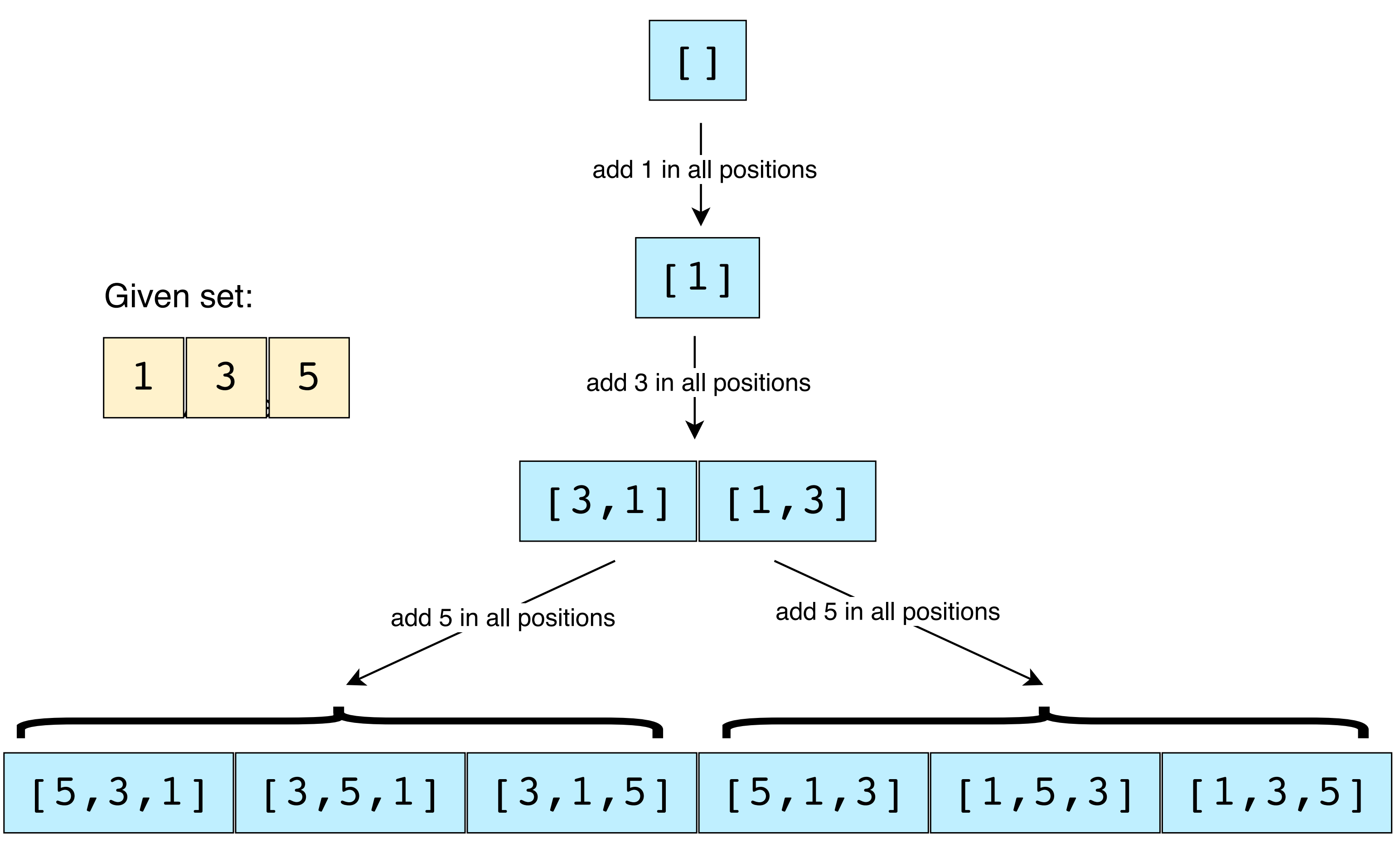
1. If the given set is empty then we have only an empty permutation set: []
2. Let's add the first element (1), the permutations will be: [1]
3. Let's add the second element (3), the permutations will be: [3,1], [1,3]
4. Let's add the third element (5), the permutations will be: [5,3,1], [3,5,1], [3,1,5], [5,1,3], [1,5,3], [1,3,5]

Let's analyze the permutations in the 3rd and 4th step. How can we generate permutations in the 4th step from the permutations of the 3rd step?

If we look closely, we will realize that when we add a new number (5), we take each permutation of the previous step and insert the new number in every position to generate the new permutations. For example, inserting '5' in different positions of [3,1] will give us the following permutations:

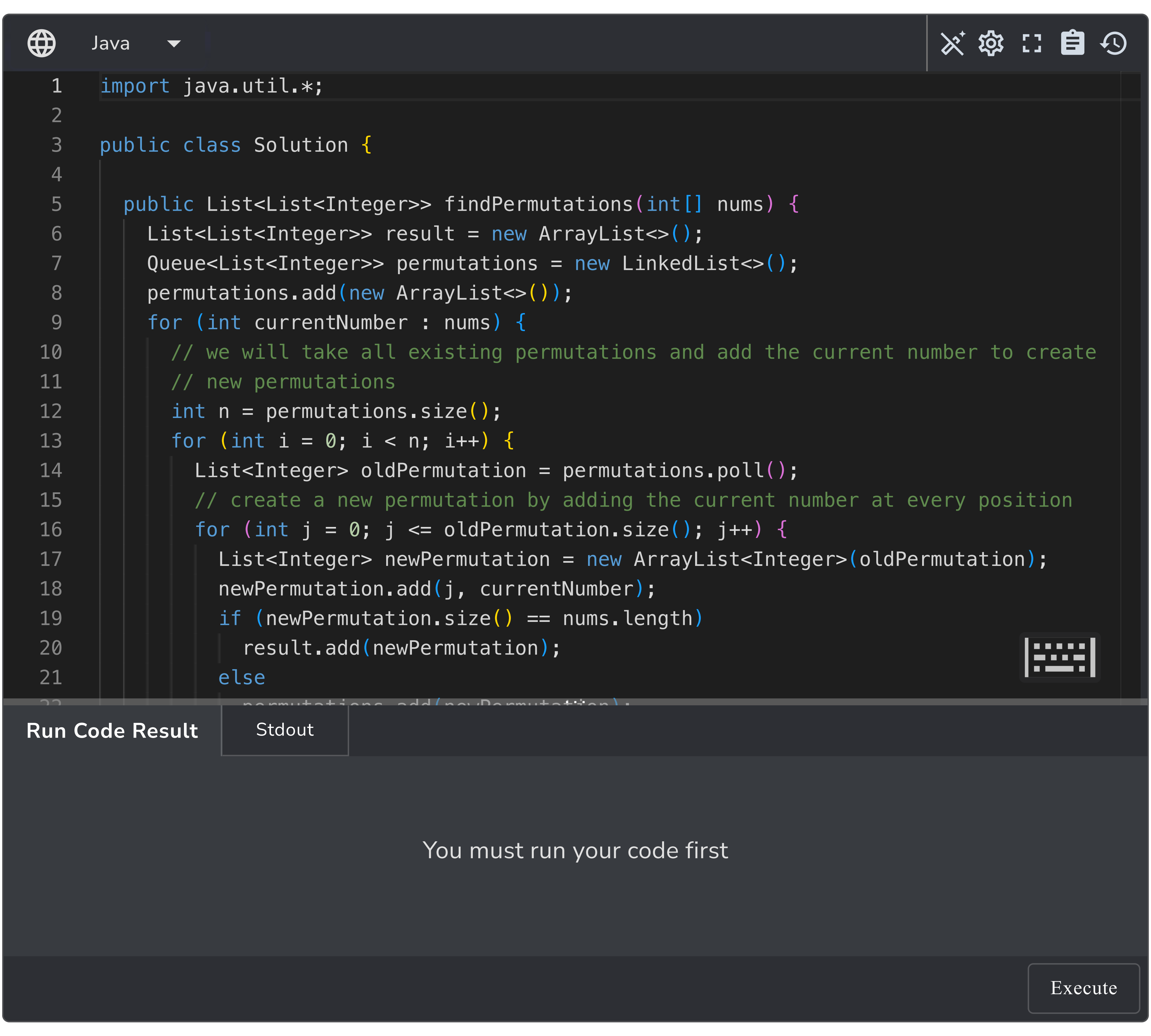
1. Inserting '5' before '3': [5,3,1]
2. Inserting '5' between '3' and '1': [3,5,1]
3. Inserting '5' after '1': [3,1,5]

Here is the visual representation of this algorithm:



Code

Here is what our algorithm will look like:



```java
1  import java.util.*;
2
3  public class Solution {
4
5    public List<List<Integer>> findPermutations(int[] nums) {
6      List<List<Integer>> result = new ArrayList<>();
7      Queue<List<Integer>> permutations = new LinkedList<>();
8      permutations.add(new ArrayList<>());
9      for (int currentNumber : nums) {
10       // we will take all existing permutations and add the current number to create
11       // new permutations
12       int n = permutations.size();
13       for (int i = 0; i < n; i++) {
14         List<Integer> oldPermutation = permutations.poll();
15         // create a new permutation by adding the current number at every position
16         for (int j = 0; j <= oldPermutation.size(); j++) {
17           List<Integer> newPermutation = new ArrayList<Integer>(oldPermutation);
18           newPermutation.add(j, currentNumber);
19           if (newPermutation.size() == nums.length)
20             result.add(newPermutation);
21           else
```

You must run your code first

Time Complexity

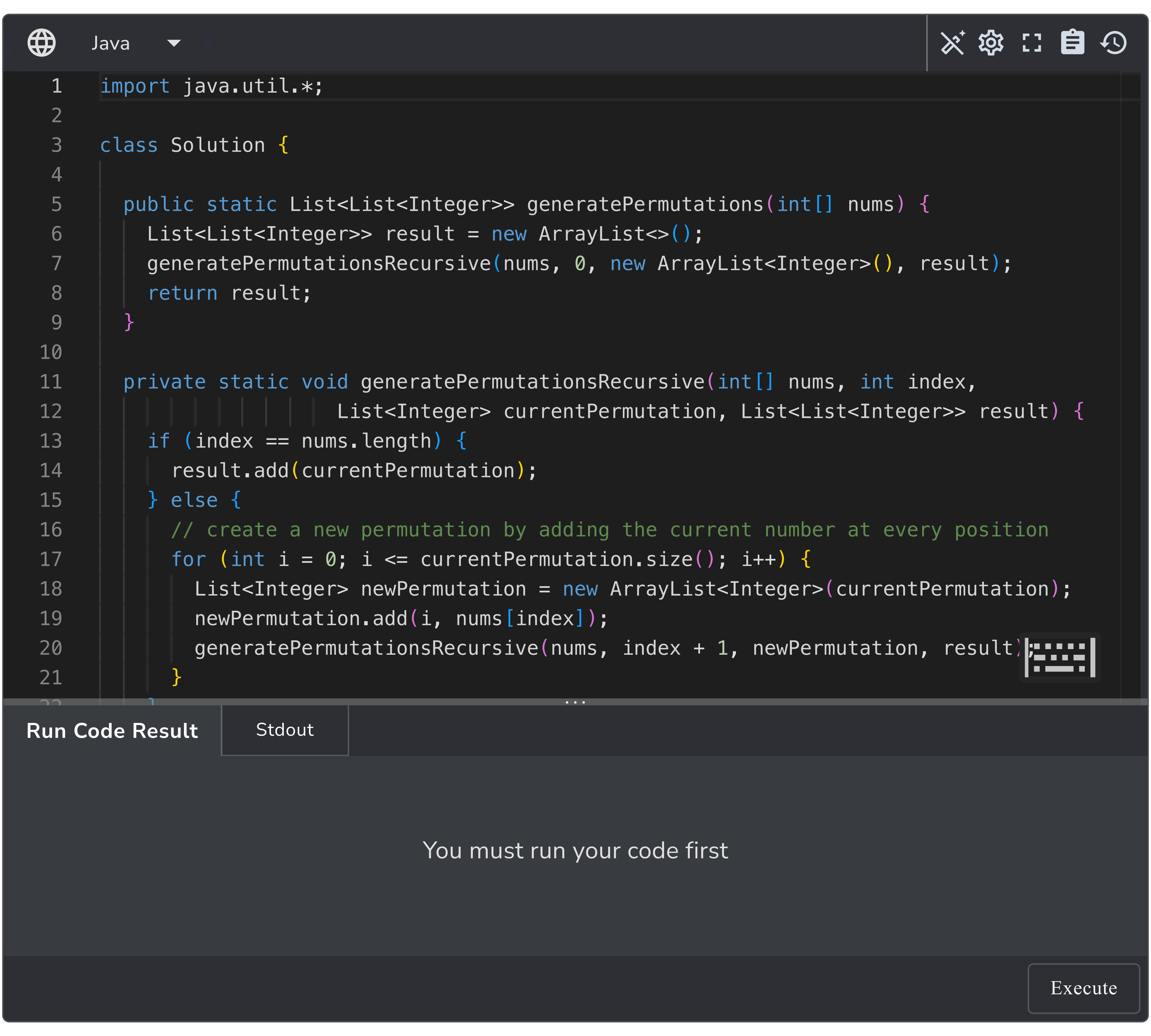
We know that there are a total of N! permutations of a set with 'N' numbers. In the algorithm above, we are iterating through all of these permutations with the help of the two 'for' loops. In each iteration, we go through all the current permutations to insert a new number in them on line 17 (line 23 for C++ solution). To insert a number into a permutation of size 'N' will take O(N), which makes the overall time complexity of our algorithm O(N*N!).

Space Complexity

All the additional space used by our algorithm is for the result list and the queue to store the intermediate permutations. If you see closely, at any time, we don't have more than N! permutations between the result list and the queue. Therefore the overall space complexity to store N! permutations each containing N elements will be O(N*N!).

Recursive Solution

Here is the recursive algorithm following a similar approach:



```java
import java.util.*;

class Solution {

  public static List<List<Integer>> generatePermutations(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    generatePermutationsRecursive(nums, 0, new ArrayList<Integer>(), result);
    return result;
  }

  private static void generatePermutationsRecursive(int[] nums, int index,
                                List<Integer> currentPermutation, List<List<Integer>> result) {
    if (index == nums.length) {
      result.add(currentPermutation);
    } else {
      // create a new permutation by adding the current number at every position
      for (int i = 0; i <= currentPermutation.size(); i++) {
        List<Integer> newPermutation = new ArrayList<Integer>(currentPermutation);
        newPermutation.add(i, nums[index]);
        generatePermutationsRecursive(nums, index + 1, newPermutation, result
      }
```

You must run your code first